

Claude Code in Action

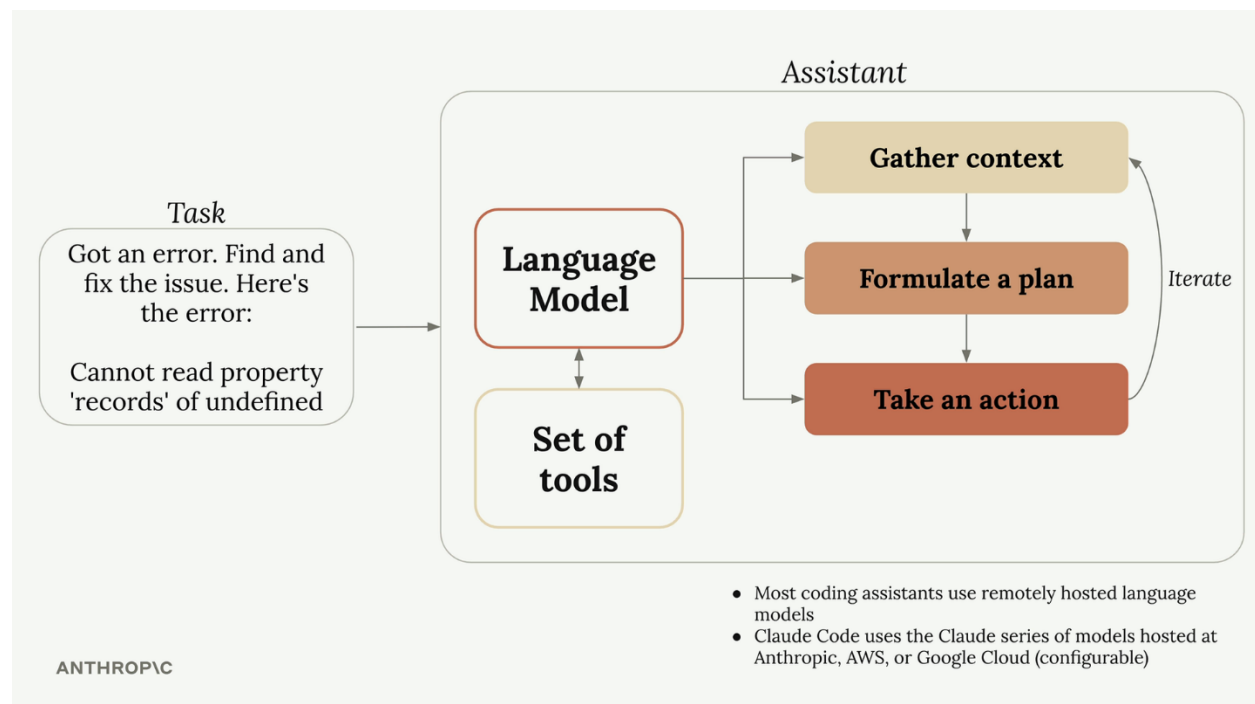
What is Claude Code?

What is a coding assistant?

A coding assistant is more than just a tool that writes code - it's a sophisticated system that uses language models to tackle complex programming tasks. Understanding how these assistants work behind the scenes will help you appreciate what makes a truly powerful coding companion.

How Coding Assistants Work

When you give a coding assistant a task, like fixing a bug based on an error message, it follows a process similar to how a human developer would approach the problem:



1. Gather context - Understanding what the error refers to, which part of the codebase is affected, and what files are relevant
2. Formulate a plan - Deciding how to solve the issue, such as changing code and running tests to verify the fix
3. Take action - Actually implementing the solution by updating files and running commands

The key insight here is that the first and last steps require the assistant to interact with the outside world - reading files, fetching documentation, running commands, or editing code.

The Tool Use Challenge

Here's where things get interesting. Language models by themselves can only process text and return text - they can't actually read files or run commands. If you ask a standalone language model to read a file, it will tell you it doesn't have that capability.

So how do coding assistants solve this problem? They use a clever system called "tool use."

How Tool Use Works

When you send a request to a coding assistant, it automatically adds instructions to your message that teach the language model how to request actions. For example, it might add text like: "If you want to read a file, respond with 'ReadFile: name of file'"

Here's the complete flow:

1. You ask: "What code is written in the main.go file?"
2. The coding assistant adds tool instructions to your request
3. The language model responds: "ReadFile: main.go"
4. The coding assistant reads the actual file and sends its contents back to the model
5. The language model provides a final answer based on the file contents

This system allows language models to effectively "read files," "write code," and "run commands" even though they're really just generating carefully formatted text responses.

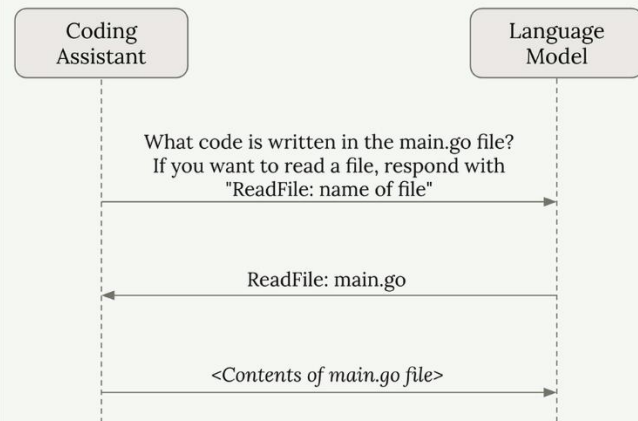
Why Claude's Tool Use Matters

Not all language models are equally good at using tools. The Claude series of models are particularly strong at understanding what tools do and using them effectively to complete complex tasks.

Tool Use

- Models are given plain text directions on how to respond a certain way to use a 'tool'
- When the model responds with a request to use a tool, the coding assistant does whatever the tool is supposed to do (read a file, write a file, make a request, etc)
- **The Claude series of models (Opus, Sonnet, Haiku) are particularly strong at understanding what tools do and using them to complete tasks**

ANTHROPIC



This strength in tool use provides several key benefits for Claude Code:

Benefits of Strong Tool Use

- Tackles harder tasks - Claude can combine different tools to handle complex work and will use tools it hasn't seen before
- Extensible platform - You can easily add new tools to Claude Code, and Claude will adapt to use them as your workflow evolves
- Better security - Claude Code can navigate codebases without requiring indexing, which often means not sending your entire codebase to external servers

Key Takeaways

Understanding coding assistants comes down to a few essential points:

- Coding assistants use language models to complete different tasks
- Language models need tools to handle most real-world programming tasks
- Not all language models use tools with the same skill level
- Claude's strong tool use enables better security, customization, and longevity in Claude Code

This tool-use capability is what transforms a simple text-generating model into a powerful coding assistant that can read your files, understand your codebase, and make meaningful changes to your projects.

Claude Code in action

Claude Code comes with a comprehensive set of built-in tools that handle common development tasks like reading files, writing code, running commands, and managing directories. But what makes Claude Code truly powerful is how intelligently it combines these tools to tackle complex, multi-step problems.

Getting hands on

Time to get Claude Code set up locally!

You can find full setup instructions here: <https://code.claude.com/docs/en/quickstart>

In short, you'll need to do the following:

1. Install Claude Code
 - a. MacOS, Linux, WSL: `curl -fsSL https://claude.ai/install.sh | bash`
 - b. Windows PowerShell: `irm https://claude.ai/install.ps1 | iex`
 - c. Windows Command Prompt (cmd.exe): `curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd`
 - d. MacOS (Homebrew): `brew install --cask claude-code`
2. After installation, run `claude` at your terminal. The first time you run this command you will be prompted to pick a color theme for the terminal and authenticate with your `claude.ai` credentials

If you get an error that `claude` isn't found after installing, or you hit a network or permissions error, see [Troubleshoot installation issues](#) in the docs.

Using Claude Code through Amazon Bedrock, Google Cloud Vertex AI, or Microsoft Foundry? See [third-party provider setup](#) for additional setup instructions.

Project setup

Working with Claude Code is more interesting if you have a project to work with.

I've put together a small project to explore with Claude Code. It is the same UI generation app shown in a previous video. Note: you don't have to run this project. You can always follow along with the remainder of the course with your own code base if you wish!

Setup

This project requires a small amount of setup:

1. Ensure you have Node JS installed locally. [Link to installation directions](#).
2. Download the zip file called uigen.zip attached to this lecture and extract it
3. In the project directory, run `npm run setup` to install dependencies and set up a local SQLite database
4. Optional: this project uses Claude through the Anthropic API to generate UI components. If you want to fully test out the app, you will need to provide an API key to access the Anthropic API. *You can skip this, and the app will still generate some static fake code.* Here's how you can set the API key:
 - a. Get an Anthropic API key at <https://console.anthropic.com/>
 - b. Place your API key in the `.env` file. Replace the literal text `your-api-key-` here with your key from the Anthropic console.
5. Start the project by running `npm run dev`

Downloads



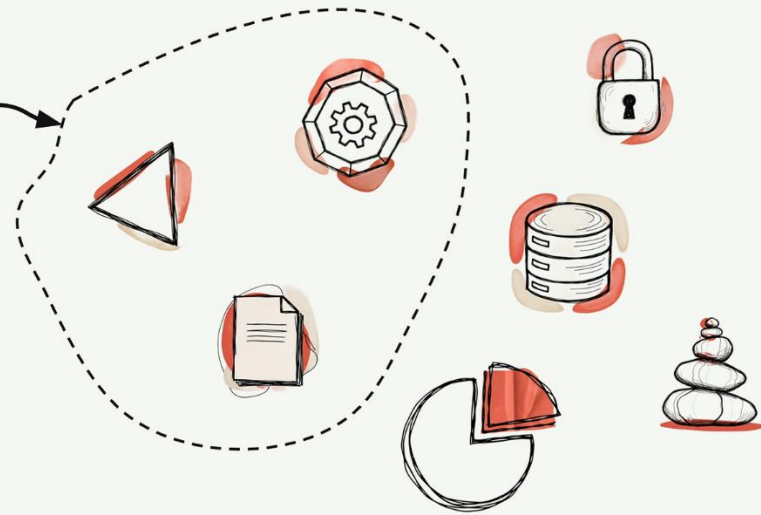
uigen.zip

Adding context

*Note: the video above shows an older # "memory mode" shortcut that has been removed. Follow the text below, which uses **/memory** and direct `CLAUDE.md` edits instead.*

When working with Claude on coding projects, context management is crucial. Your project might have dozens or hundreds of files, but Claude only needs the right information to help you effectively. Too much irrelevant context actually decreases Claude's performance, so learning to guide it toward relevant files and documentation is essential.

What Claude
needs to know to
accomplish your task



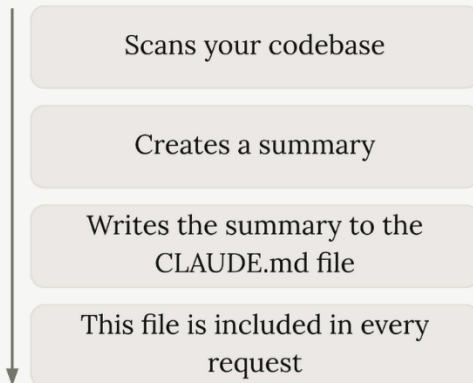
ANTHROPIC

The `/init` Command

When you first start Claude in a new project, run the **`/init`** command. This tells Claude to analyze your entire codebase and understand:

- The project's purpose and architecture
- Important commands and critical files
- Coding patterns and structure

The `/init` command



```
> /init is analyzing your codebase...

• I'll analyze your codebase to understand its structure
  and create a comprehensive CLAUDE.md file. Let me
  start by exploring the repository.

* Jiving... (6s · 68 tokens · esc to interrupt)

> 
? for shortcuts
```

After analyzing your code, Claude creates a summary and writes it to a **CLAUDE.md** file. When Claude asks for permission to create this file, you can either hit Enter to approve each write operation, or press Shift+Tab to let Claude write files freely throughout your session.

The **CLAUDE.md** File

The **CLAUDE.md** file serves two main purposes:

- Guides Claude through your codebase, pointing out important commands, architecture, and coding style
- Allows you to give Claude specific or custom directions

This file gets included in every request you make to Claude, so it's like having a persistent system prompt for your project.

CLAUDE.md File Locations

Claude recognizes three different **CLAUDE.md** files in three common locations:

CLAUDE.md files

Three common locations

CLAUDE.md

- Generated with `/init`
- Commit to source control
- Shared with other engineers

CLAUDE.local.md

- Not shared with other engineers
- Contains personal instructions and customizations for Claude

~/.claude/CLAUDE.md

- Used with all projects on your machine
- Contains instructions that you want Claude to follow on all projects

ANTHROPIC

- CLAUDE.md - Generated with `/init`, committed to source control, shared with other engineers
- CLAUDE.local.md - Not shared with other engineers, contains personal instructions and customizations for Claude
- ~/.claude/CLAUDE.md - Used with all projects on your machine, contains instructions that you want Claude to follow on all projects

Adding Custom Instructions

You can customize how Claude behaves by adding instructions to your **CLAUDE.md** file. For example, if Claude is adding too many comments to code, you can address this by updating the file. Edit **CLAUDE.md** directly in your editor, or run `/memory` inside Claude Code to open the file. Add an instruction like **Use comments sparingly. Only comment complex code.**

Claude reads this file at the start of every conversation, so changes apply to your next message.

File Mentions with '@'

When you need Claude to look at specific files, use the `@` symbol followed by the file path. This automatically includes that file's contents in your request to Claude.

For example, if you want to ask about your authentication system and you know the relevant files, you can type:

How does the auth system work? @auth

Claude will show you a list of auth-related files to choose from, then include the selected file in your conversation.

Referencing Files in **CLAUDE.md**

You can also mention files directly in your **CLAUDE.md** file using the same @ syntax. This is particularly useful for files that are relevant to many aspects of your project.

For example, if you have a database schema file that defines your data structure, you might add this to your **CLAUDE.md**:

The database schema is defined in the @prisma/schema.prisma file. Reference it anytime you need to understand the structure of data stored in the database.

When you mention a file this way, its contents are automatically included in every request, so Claude can answer questions about your data structure immediately without having to search for and read the schema file each time.

This is also useful if your repo already has an **AGENTS.md** file for another tool. You don't need to duplicate the instructions; add @**AGENTS.md** on the first line of your **CLAUDE.md**, and Claude will load the contents of that file first. Then, you can add any Claude-specific instructions below the import.

Making changes

*Note: the video above shows older "think harder" keywords that no longer have an effect. Follow the text below, which uses /**effort**.*

When working with Claude in your development environment, you'll often need to make changes to existing projects. This guide covers practical techniques for implementing changes effectively, including visual communication with screenshots and leveraging Claude's advanced reasoning capabilities.

Using Screenshots for Precise Communication

One of the most effective ways to communicate with Claude is through screenshots. When you want to modify a specific part of your interface, taking a screenshot helps Claude understand exactly what you're referring to.

To paste a screenshot into Claude, use **Ctrl+V** (not **Cmd+V** on macOS). This keyboard shortcut is specifically designed for pasting screenshots into the chat interface. Once you've pasted the image, you can ask Claude to make specific changes to that area of your application.

Planning Mode

For more complex tasks that require extensive research across your codebase, you can enable Planning Mode. This feature makes Claude do thorough exploration of your project before implementing changes.

Enable Planning Mode by typing **/plan** or by pressing **Shift + Tab** twice (or once if you're already auto-accepting edits). In this mode, Claude will:

- Read more files in your project
- Create a detailed implementation plan
- Show you exactly what it intends to do
- Wait for your approval before proceeding

This gives you the opportunity to review the plan and redirect Claude if it missed something important or didn't consider a particular scenario.

Tip: when reviewing the plan, you can press **Ctrl+G** to open it in your text editor. You can precise edits before approving the plan, and Claude will see the final version you submit.

Effort level: how hard Claude thinks

By default, Claude reasons through problems before answering. You'll see hints like "still thinking" while it works. If you want to see Claude's reasoning process, press **Ctrl+O** to expand the actual reasoning steps.

You can control how Claude reasons through a problem by setting an effort level.

Run **/effort** to see your current level and adjust it: **low** is faster and cheaper, **max** reasons longest on hard problems. The default depends on your model and plan — **/effort** shows you what yours is.

If you want to signal to Claude that it should do extra thinking on a single prompt, use the keyword **ultrathink** in your prompt. This signals to Claude that it should reason more on this turn, but doesn't adjust the session's effort level.

When to Use Planning vs Effort

These two features handle different types of complexity:

Planning Mode is best for:

- Tasks requiring broad understanding of your codebase
- Multi-step implementations
- Changes that affect multiple files or components

Adjusting to a higher effort level is best for:

- Complex logic problems
- Debugging difficult issues
- Algorithmic challenges

You can combine both modes for tasks that require both breadth and depth. Just keep in mind that both features consume additional tokens, so there's a cost consideration for using them.

Controlling context

*Note: the video above shows an older # shortcut for adding memories. Follow the text below, which uses **/memory** instead.*

When working with Claude on complex tasks, you'll often need to guide the conversation to keep it focused and productive. There are several techniques you can use to control the flow of your conversation and help Claude stay on track.

Interrupting Claude with Escape

Sometimes Claude starts heading in the wrong direction or tries to tackle too much at once. You can press the Escape key to stop Claude mid-response, allowing you to redirect the conversation.

This is particularly useful when you want Claude to focus on one specific task instead of trying to handle multiple things simultaneously. For example, if you ask Claude to write

tests for multiple functions and it starts creating a comprehensive plan for all of them, you can interrupt and ask it to focus on just one function at a time.

Combining Escape with Memories

One of the most powerful applications of the escape technique is fixing repetitive errors. When Claude makes the same mistake repeatedly across different conversations, you can:

- Press Escape to stop the current response
- Run **/memory** (or edit **CLAUDE.md** directly) to add a note about the correct approach
- Continue the conversation with the corrected information

This prevents Claude from making the same error in future conversations on your project.

Rewinding Conversations

During long conversations, you might accumulate context that becomes irrelevant or distracting. For instance, if Claude encounters an error and spends time debugging it, that back-and-forth discussion might not be useful for the next task.

You can rewind the conversation by pressing Escape twice or typing **/rewind**. This shows you all the messages you've sent, allowing you to jump back to an earlier point and continue from there. This technique helps you:

- Maintain valuable context (like Claude's understanding of your codebase)
- Remove distracting or irrelevant conversation history
- Keep Claude focused on the current task

Context Management Commands

Claude provides several commands to help manage conversation context effectively:

/compact

The **/compact** command summarizes your entire conversation history while preserving the key information Claude has learned. This is ideal when:

- Claude has gained valuable knowledge about your project
- You want to continue with related tasks
- The conversation has become long but contains important context

Use **compact** when Claude has learned a lot about the current task and you want to maintain that knowledge as it moves to the next related task.

/clear

The **/clear** command starts a new conversation with fresh context. This is most useful when:

- You're switching to a completely different, unrelated task
- The current conversation context might confuse Claude for the new task
- You want to start over without any previous context

You can still go back to the previous conversation later with **/resume**. The **/clear** command does not remove the conversation from your session history.

When to Use These Techniques

These conversation control techniques are particularly valuable during:

- Long-running conversations where context can become cluttered
- Task transitions where previous context might be distracting
- Situations where Claude repeatedly makes the same mistakes
- Complex projects where you need to maintain focus on specific components

By using **escape**, **double-tap escape**, **/compact**, and **/clear** strategically, you can keep Claude focused and productive throughout your development workflow. These aren't just convenience features—they're essential tools for maintaining effective AI-assisted development sessions.

Custom commands

Claude Code comes with built-in commands that you can access by typing a forward slash, but you can also create your own custom commands to automate repetitive tasks you run frequently.

Creating Custom Commands

To create a custom command, you need to set up a specific folder structure in your project:

1. Find the **.claude** folder in your project directory
2. Create a new directory called **commands** inside it
3. Create a new markdown file with your desired command name (like **audit.md**)

The filename becomes your command name - so **audit.md** creates the **/audit** command.

Example: Audit Command

Here's a practical example of a custom command that audits project dependencies for vulnerabilities:

This audit command does three things:

1. Runs **npm audit** to find vulnerable installed packages
2. Runs **npm audit fix** to apply updates
3. Runs tests to verify the updates didn't break anything

After creating your command file, Claude Code picks it up automatically. You don't need to restart.

Commands with Arguments

Custom commands can accept arguments using the **\$ARGUMENTS** placeholder. This makes them much more flexible and reusable.

For example, a **write_tests.md** command might contain:

Write comprehensive tests for: \$ARGUMENTS

Testing conventions:

- * Use Vitest with React Testing Library
- * Place test files in a `__tests__` directory in the same folder as the source file
- * Name test files as `[filename].test.ts(x)`
- * Use `@/` prefix for imports

Coverage:

- * Test happy paths

* Test edge cases

* Test error states

You can then run this command with a file path:

/write_tests the use-auth.ts file in the hooks directory

The arguments don't have to be file paths - they can be any string you want to pass to give Claude context and direction for the task.

Key Benefits

- Automation - Turn repetitive workflows into single commands
- Consistency - Ensure the same steps are followed every time
- Context - Provide Claude with specific instructions and conventions for your project
- Flexibility - Use arguments to make commands work with different inputs

Custom commands are particularly useful for project-specific workflows like running test suites, deploying code, or generating boilerplate following your team's conventions.

MCP servers with Claude Code

You can extend Claude Code's capabilities by adding MCP (Model Context Protocol) servers. These servers run either remotely or locally on your machine and provide Claude with new tools and abilities it wouldn't normally have.

One of the most popular MCP servers is Playwright, which gives Claude the ability to control a web browser. This opens up powerful possibilities for web development workflows.

Installing the Playwright MCP Server

To add the Playwright server to Claude Code, run this command in your terminal (not inside Claude Code):

```
claude mcp add playwright npx @playwright/mcp@latest
```

This command does two things:

- Names the MCP server "playwright"
- Provides the command that starts the server locally on your machine

Managing Permissions

When you first use MCP server tools, Claude will ask for permission each time. If you get tired of these permission prompts, you can pre-approve the server by editing your settings.

Open the **.claude/settings.local.json** file and add the server to the allow array:

```
{  
  "permissions": {  
    "allow": ["mcp__playwright"],  
    "deny": []  
  }  
}
```

Note the double underscores in **mcp__playwright**. This allows Claude to use the Playwright tools without asking for permission every time.

Practical Example: Improving Component Generation

Here's a real-world example of how the Playwright MCP server can improve your development workflow. Instead of manually testing and tweaking prompts, you can have Claude:

1. Open a browser and navigate to your application
2. Generate a test component
3. Analyze the visual styling and code quality
4. Update the generation prompt based on what it observes
5. Test the improved prompt with a new component

For instance, you might ask Claude to:

"Navigate to localhost:3000, generate a basic component, review the styling, and update the generation prompt at @src/lib/prompts/generation.tsx to produce better components going forward."

Claude will use the browser tools to interact with your app, examine the generated output, and then modify your prompt file to encourage more original and creative designs.

Results and Benefits

In practice, this approach can lead to significantly better results. Instead of generic purple-to-blue gradients and standard Tailwind patterns, Claude might update prompts to encourage:

- Warm sunset gradients (orange-to-pink-to-purple)
- Ocean depth themes (teal-to-emerald-to-cyan)
- Asymmetric designs and overlapping elements
- Creative spacing and unconventional layouts

The key advantage is that Claude can see the actual visual output, not just the code, which allows it to make much more informed decisions about styling improvements.

Exploring Other MCP Servers

Playwright is just one example of what's possible with MCP servers. The ecosystem includes servers for:

- Database interactions
- API testing and monitoring
- File system operations
- Cloud service integrations
- Development tool automation

Consider exploring MCP servers that align with your specific development needs. They can transform Claude from a code assistant into a comprehensive development partner that can interact with your entire toolchain.

Github integration

Claude Code offers an official GitHub integration that lets Claude run inside GitHub Actions. This integration provides two main workflows: mention support for issues and pull requests, and automatic pull request reviews.

Setting Up the Integration

To get started, run `/install-github-app` in Claude. This command walks you through the setup process:

- Install the Claude Code app on GitHub

- Add your API key
- Automatically generate a pull request with the workflow files

The generated pull request adds two GitHub Actions to your repository. Once merged, you'll have the workflow files in your **.github/workflows** directory.

Default GitHub Actions

The integration provides two main workflows:

Mention Action

You can mention Claude in any issue or pull request using **@claude**. When mentioned, Claude will:

- Analyze the request and create a task plan
- Execute the task with full access to your codebase
- Respond with results directly in the issue or PR

Pull Request Action

Whenever you create a pull request, Claude automatically:

- Reviews the proposed changes
- Analyzes the impact of modifications
- Posts a detailed report on the pull request

Customizing the Workflows

After merging the initial pull request, you can customize the workflow files to fit your project's needs. Here's how to enhance the mention workflow:

Adding Project Setup

Before Claude runs, you can add steps to prepare your environment:

- name: Project Setup

run: |

npm run setup

npm run dev:daemon

Custom Instructions

Provide Claude with context about your project setup:

custom_instructions: |

The project is already set up with all dependencies installed.

The server is already running at localhost:3000. Logs from it are being written to logs.txt. If needed, you can query the db with the 'sqlite3' cli. If needed, use the mcp__playwright set of tools to launch a browser and interact with the app.

MCP Server Configuration

You can configure MCP servers to give Claude additional capabilities:

mcp_config: |

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": [
        "@playwright/mcp@latest",
        "--allowed-origins",
        "localhost:3000;cdn.tailwindcss.com;esm.sh"
      ]
    }
  }
}
```

Tool Permissions

When running Claude in GitHub Actions, you must explicitly list all allowed tools. This is especially important when using MCP servers.

allowed_tools:

"Bash(npm:*),Bash(sqlite3:*),mcp__playwright__browser_snapshot,mcp__playwright__br

rowser_click,..."

Unlike local development, there's no shortcut for permissions in GitHub Actions. Each tool from each MCP server must be individually listed.

Best Practices

When setting up Claude's GitHub integration:

- Start with the default workflows and customize gradually
- Use custom instructions to provide project-specific context
- Be explicit about tool permissions when using MCP servers
- Test your workflows with simple tasks before complex ones
- Consider your project's specific needs when configuring additional steps

The GitHub integration transforms Claude from a development assistant into an automated team member that can handle tasks, review code, and provide insights directly within your GitHub workflow.

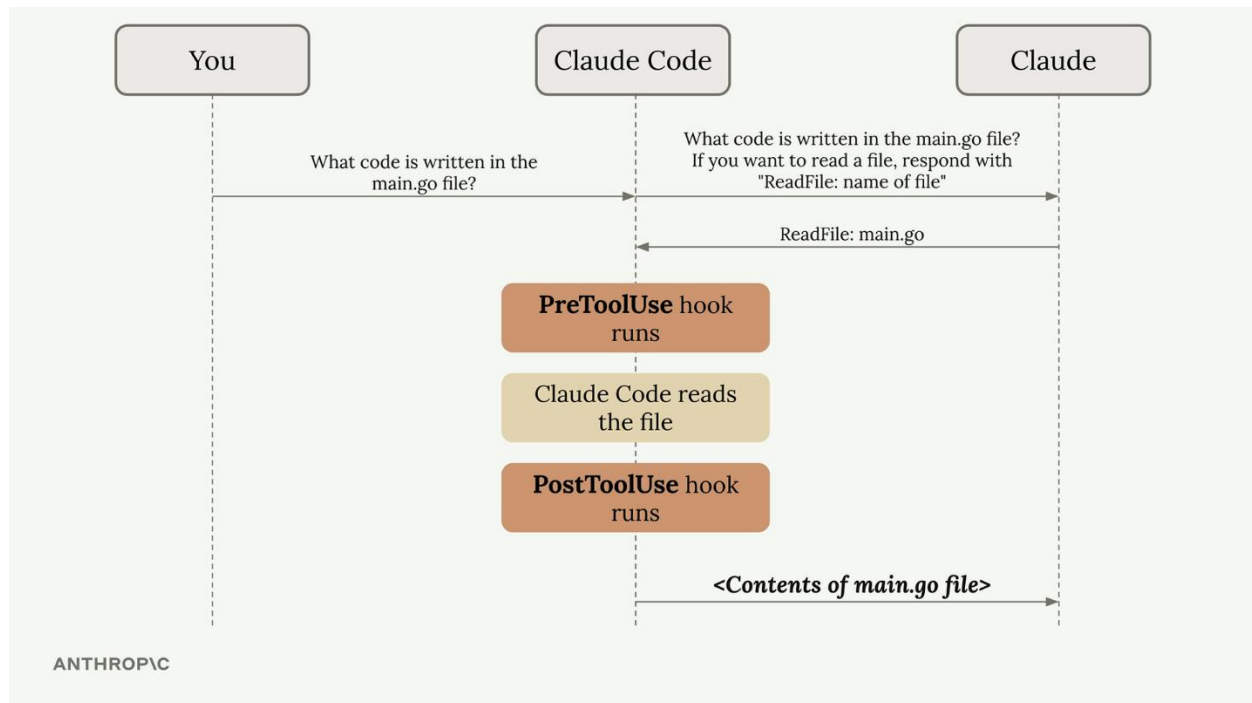
Hooks and the SDK

Introducing hooks

How Hooks Work

To understand hooks, let's first review the normal flow when you interact with Claude Code. When you ask Claude something, your query gets sent to the Claude model along with tool definitions. Claude might decide to use a tool by providing a formatted response, and then Claude Code executes that tool and returns the result.

Hooks insert themselves into this process, allowing you to execute code just before or just after the tool execution happens.



Two of the most common hook types are below (you'll see other hooks in a later lesson):

- PreToolUse hooks - Run before a tool is called
- PostToolUse hooks - Run after a tool is called

Hook Configuration

Hooks are defined in Claude settings files. You can add them to:

- Global - `~/.claude/settings.json` (affects all projects)
- Project - `.claude/settings.json` (shared with team)
- Project (not committed) - `.claude/settings.local.json` (personal settings)

You can write hooks by hand in these files or use the `/hooks` command inside Claude Code.

Hook Definitions

Defined in...

Global	~/.claude/settings.json
Project	.claude/settings.json
Project (not committed)	.claude/settings.local.json

Write **by hand** or by using the `/hooks` command

ANTHROPIC

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/read_hook.ts"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/edit_hook.ts"
          }
        ]
      }
    ]
  }
}
```

The configuration structure includes two main sections:

Hooks that run **before** a tool is called

Hooks that **after** before a tool is called

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/read_hook.ts"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/edit_hook.ts"
          }
        ]
      }
    ]
  }
}
```

ANTHROPIC

PreToolUse Hooks

PreToolUse hooks run before a tool is executed. They include a matcher that specifies which tool types to target:

```
"PreToolUse": [  
  {  
    "matcher": "Read",  
    "hooks": [  
      {  
        "type": "command",  
        "command": "node /home/hooks/read_hook.js"  
      }  
    ]  
  }  
]
```

Before the 'Read' tool is executed, this configuration runs the specified command. Your command receives details about the tool call Claude wants to make, and you can:

- Allow the operation to proceed normally
- Block the tool call and send an error message back to Claude

PostToolUse Hooks

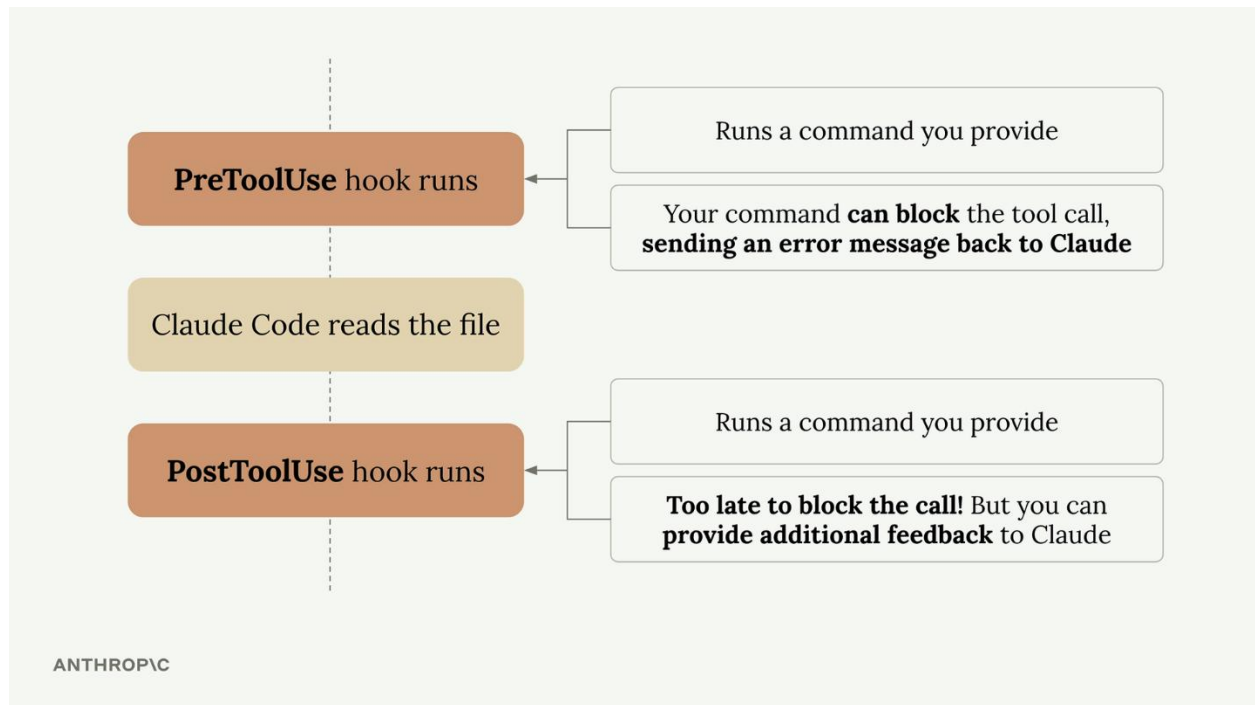
PostToolUse hooks run after a tool has been executed. Here's an example that triggers after write, edit, or multi-edit operations:

```
"PostToolUse": [  
  {  
    "matcher": "Write|Edit",  
    "hooks": [  
      {  
        "type": "command",  
        "command": "node /home/hooks/edit_hook.js"  
      }  
    ]  
  }  
]
```

```
}  
]
```

Since the tool call has already occurred, PostToolUse hooks can't block the operation. However, they can:

- Run follow-up operations (like formatting a file that was just edited)
- Provide additional feedback to Claude about the tool use



Practical Applications

Here are some common ways to use hooks:

- Code formatting - Automatically format files after Claude edits them
- Testing - Run tests automatically when files are changed
- Access control - Block Claude from reading or editing specific files
- Code quality - Run linters or type checkers and provide feedback to Claude
- Logging - Track what files Claude accesses or modifies
- Validation - Check naming conventions or coding standards

The key insight is that hooks let you extend Claude Code's capabilities by integrating your own tools and processes into the workflow. PreToolUse hooks give you control over what Claude can do, while PostToolUse hooks let you enhance what Claude has done.

Defining hooks



queries.zip

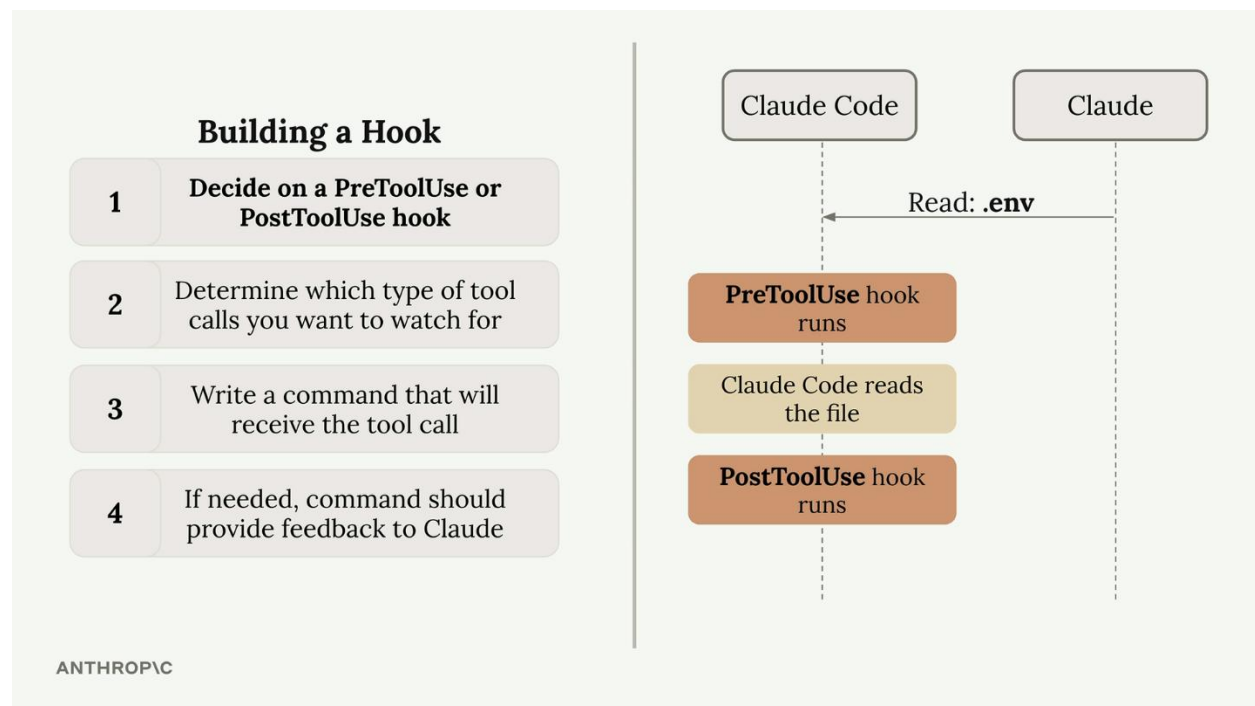


queries_COMPLETED.zip

Hooks in Claude Code allow you to intercept and control tool calls before or after they execute. This gives you fine-grained control over what Claude can and cannot do in your development environment.

Building a Hook

Creating a hook involves four main steps:



1. Decide on a PreToolUse or PostToolUse hook - PreToolUse hooks can prevent tool calls from executing, while PostToolUse hooks run after the tool has already been used
2. Determine which type of tool calls you want to watch for - You need to specify exactly which tools should trigger your hook

3. Write a command that will receive the tool call - This command gets JSON data about the proposed tool call via standard input
4. If needed, command should provide feedback to Claude - Your command's exit code tells Claude whether to allow or block the operation

Available Tools

Claude Code provides several built-in tools that you can monitor with hooks:

Building a Hook

- 1 Decide on a PreToolUse or PostToolUse hook
- 2 **Determine which type of tool calls you want to watch for**
- 3 Write a command that will receive the tool call
- 4 If needed, command should provide feedback to Claude

Tool Name	Purpose
Read	Read a file
Edit, MultiEdit	Edit an existing file
Write	Create a file and write to it
Bash	Execute a command
Glob	Find files/folders based upon a pattern
Grep	Search for content
Task	Create a sub-agent to complete a particular task
WebFetch, WebSearch	Search or fetch a particular page

ANTHROPIC

To see exactly which tools are available in your current setup, you can ask Claude directly for a list. This is especially useful since the available tools can change when you add custom MCP servers.

Tool Call Data Structure

When your hook command executes, Claude sends JSON data through standard input containing details about the proposed tool call:

Building a Hook

- 1 Decide on a PreToolUse or PostToolUse hook
- 2 Determine which type of tool calls you want to watch for
- 3 **Write a command that will receive the tool call**
- 4 If needed, command should provide feedback to Claude

ANTHROPIC

Tool Call Data

```
{
  "session_id": "2d6a1e4d-6...",
  "transcript_path": "/Users/sg/...",
  "hook_event_name": "PreToolUse",
  "tool_name": "Read",
  "tool_input": {
    "file_path": "/code/queries/.env"
  }
}
```

Standard In

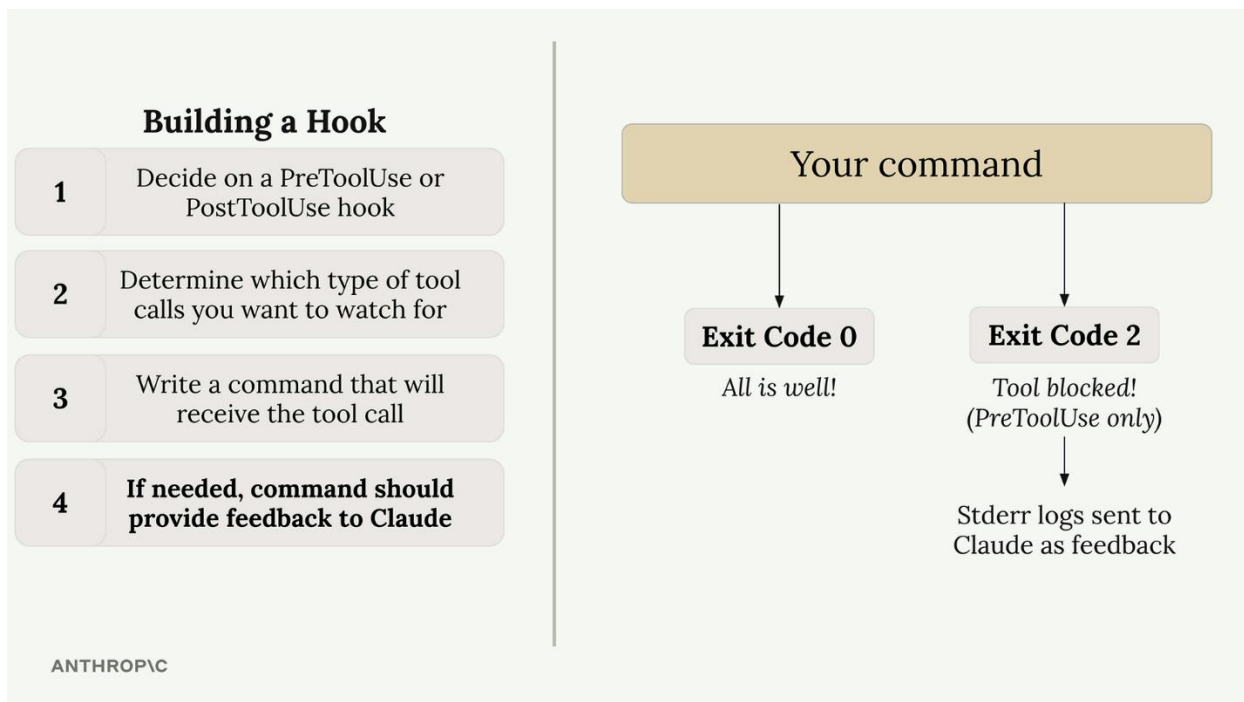
Your command

```
{
  "session_id": "2d6a1e4d-6...",
  "transcript_path": "/Users/sg/...",
  "hook_event_name": "PreToolUse",
  "tool_name": "Read",
  "tool_input": {
    "file_path": "/code/queries/.env"
  }
}
```

Your command reads this JSON from standard input, parses it, and then decides whether to allow or block the operation based on the tool name and input parameters.

Exit Codes and Control Flow

Your hook command communicates back to Claude through exit codes:



- Exit Code 0 - Everything is fine, allow the tool call to proceed
- Exit Code 2 - Block the tool call (PreToolUse hooks only)

When you exit with code 2 in a PreToolUse hook, any error messages you write to standard error will be sent to Claude as feedback, explaining why the operation was blocked.

Example Use Case

A common use case is preventing Claude from reading sensitive files like `.env` files. Since both the **Read** and **Grep** tools can access file contents, you'd want to monitor both tool types and check if they're trying to access restricted file paths.

This approach gives you complete control over Claude's file system access while providing clear feedback about why certain operations are restricted.

Implementing a hook

Note: the video above shows an earlier version of this exercise that included Grep. Follow the text below for the current version.

Let's build a hook that prevents Claude from reading sensitive files. This is a practical example of how a PreToolUse hook can intercept tool calls before they run.

What this exercise covers

You'll write a hook that blocks the Read tool from opening **.env**. This protects your environment variables during a session.

Note that this hook covers Read only. Blocking Grep or Bash from reaching the same file requires checking each tool's input shape separately, since each tool sends different fields. For comprehensive file protection, combine a hook with a **permissions.deny** rule like **"Read(**/.env)"**. See the [hooks guide](#) for a fuller treatment.

Configure the hook

Open **.claude/settings.local.json** and add a PreToolUse hook that matches the Read tool:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [
          { "type": "command", "command": "node $PWD/hooks/read_hook.js" }
        ]
      }
    ]
  }
}
```

Write the hook script

Create **hooks/read_hook.js**:

```
process.stdin.setEncoding("utf8");

let input = "";

process.stdin.on("data", (d) => (input += d));

process.stdin.on("end", () => {
  const toolArgs = JSON.parse(input);
```

```
const readPath = toolArgs.tool_input?.file_path || "";
if (readPath.includes(".env")) {
  console.error("You cannot read the .env file");
  process.exit(2);
}
process.exit(0);
});
```

The hook reads the tool call from stdin as JSON, checks **tool_input.file_path**, and exits with code 2 to block the call (anything written to stderr becomes the message Claude sees).

Test it

In a Claude Code session, ask Claude to read your **.env** file. You should see:

You cannot read the .env file

That's the hook blocking the Read call. Ask Claude to read a different file and it works normally.

Why Read only?

Each tool sends a different input shape. Read sends **{"file_path": ".."}**; Grep sends **{"pattern": "...", "path": ".."}** where **path** is a search *directory*, not a file; Bash sends **{"command": ".."}**. A check on **file_path** catches Read but won't catch a project-wide grep for **API_KEY** or a **cat .env** in Bash. To cover those, you'd write separate matchers per tool and inspect each one's specific fields — or use **permissions.deny** rules, which apply uniformly across tools.

Gotchas around hooks

Open in Claude

You may notice that after running the npm run setup command there are two settings.json files in the .claude directory. Let me explain what's going on there.

The Claude Code documentation lists some recommendations around hooks security:

Security Best Practices

Here are some key practices for writing more secure hooks:

1. **Validate and sanitize inputs** - Never trust input data blindly
2. **Always quote shell variables** - Use `"$VAR"` not `$VAR`
3. **Block path traversal** - Check for `..` in file paths
4. **Use absolute paths** - Specify full paths for scripts
5. **Skip sensitive files** - Avoid `.env` , `.git/` , keys, etc.

One of the recommendations is to use absolute paths (rather than relative paths) for scripts. This helps mitigate [path interception](#) and [binary planting](#) attacks.

This recommendation also makes it much more challenging to share settings.json files. The reason is simple: the absolute path to any of the hook scripts on your machine will likely be different from the absolute path on my machine, simply because we will probably place the project in separate directories.

To solve this problem, our project has a settings.example.json file. Inside of it, the script references contain a \$PWD placeholder. When we run npm run setup, some dependencies are installed, but it also runs an init-claude.js script placed inside the scripts directory. This script will replace those \$PWD placeholder with the absolute path to the project on your machine, copy the settings.example.json file, and rename it to settings.local.json.

This script allows us to share settings.json files but still use the recommended absolute paths!

Useful hooks!

Claude Code hooks can help address common weaknesses in AI-assisted development, particularly on larger projects. These hooks run automatically when Claude makes changes to your code, providing immediate feedback and preventing common issues.

TypeScript Type Checking Hook

One of the most useful hooks addresses a fundamental problem: when Claude modifies a function signature, it often doesn't update all the places where that function is called throughout your project.

For example, if you ask Claude to add a **verbose** parameter to a function in **schema.ts**, it will successfully update the function definition but miss the call site in **main.ts**. This creates type errors that Claude doesn't immediately catch.

The solution is a post-tool-use hook that runs the TypeScript compiler after every file edit:

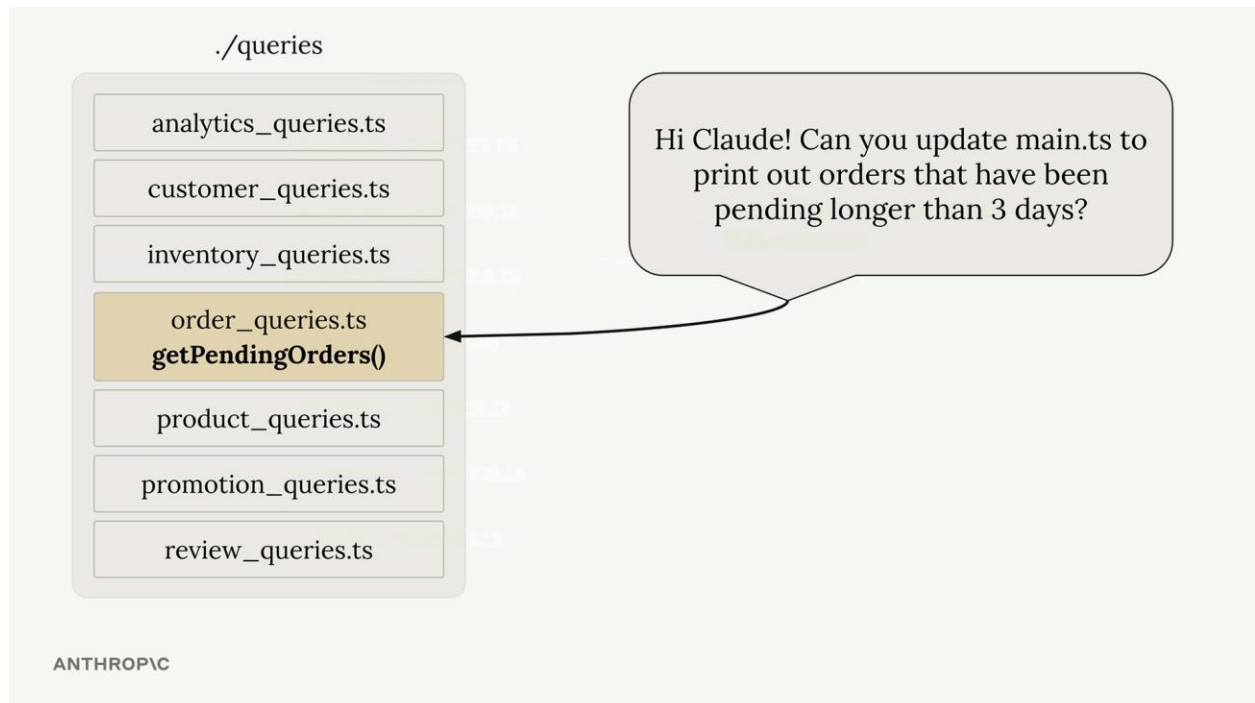
- Runs **tsc --noEmit** to check for type errors
- Captures any errors found
- Feeds the errors back to Claude immediately
- Prompts Claude to fix the issues in other files

This hook works for any typed language where you can run a type checker. For untyped languages, you could implement similar functionality using automated tests instead.

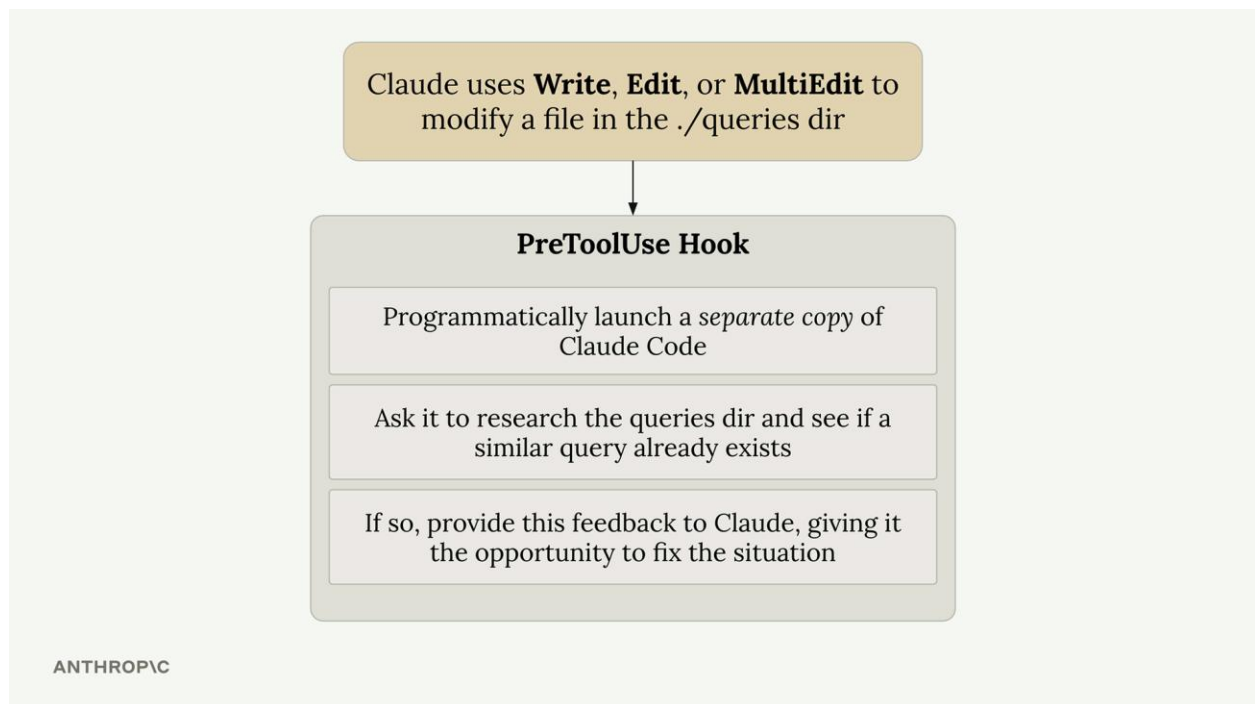
Query Duplication Prevention Hook

In larger projects with many database queries, Claude sometimes creates duplicate functionality instead of reusing existing code. This is especially problematic when you give Claude complex, multi-step tasks that include database operations as just one component.

Consider a project structure with multiple query files, each containing many SQL functions. When you ask Claude to "create a Slack integration that alerts about orders pending longer than 3 days," it might write a new query instead of using the existing **getPendingOrders()** function.



The query duplication hook addresses this by implementing a review process:



Here's how it works:

- Triggers when Claude modifies files in the **`./queries`** directory
- Launches a separate instance of Claude Code programmatically

- Asks the second instance to review the changes and check for similar existing queries
- If duplicates are found, provides feedback to the original Claude instance
- Prompts Claude to remove the duplicate and use the existing functionality

Implementation Considerations

Both hooks use the pre-tool-use or post-tool-use hook system. The TypeScript hook is relatively lightweight and runs quickly. The query duplication hook requires more resources since it launches a separate Claude instance for each review.

For the query hook, consider these trade-offs:

- Benefits: Cleaner codebase with less duplication
- Costs: Additional time and API usage for each query directory edit
- Recommendation: Only monitor critical directories to minimize overhead

The hooks use Claude's Agent SDK to programmatically interact with the AI. This allows you to create sophisticated workflows where one Claude instance can review and provide feedback on another's work.

Extending These Concepts

These hooks demonstrate broader principles you can apply to your own projects:

- Use compiler/linter output to provide immediate feedback
- Implement code review processes using separate AI instances
- Focus monitoring on high-value directories where consistency matters most
- Balance automation benefits against performance costs

The key is identifying the specific pain points in your development workflow and creating targeted hooks that address those issues automatically.

Another useful hook

Open in Claude

There are more hooks beyond the PreToolUse and PostToolUse hooks discussed in this course. There are also:

- Notification - Runs when Claude Code sends a notification, which occurs when Claude needs permission to use a tool, or after Claude Code has been idle for 60 seconds
- Stop - Runs when Claude Code has finished responding
- SubagentStop - Runs when a subagent (these are displayed as a "Task" in the UI) has finished
- PreCompact - Runs before a compact operation occurs, either manual or automatic
- UserPromptSubmit - Runs when the user submits a prompt, before Claude processes it
- SessionStart - Runs when starting or resuming a session
- SessionEnd - Runs when a session ends

Here's the confusing part:

1. The stdin input to your commands will change based upon the type of hook being executed (PreToolUse, PostToolUse, Notification, etc)
2. The tool_input contained in that will differ based upon the tool that was called (in the case of PreToolUse and PostToolUse hooks)

For example, here's a sample of some stdin input to a hook, where the hook is a PostToolUse that was watching for uses of the TodoWrite tool. For reference, that is the tool that Claude uses to keep track of to-do items.

```
{
  "session_id": "9ecf22fa-edf8-4332-ae85-b6d5456eda64",
  "transcript_path": "<path_to_transcript>",
  "hook_event_name": "PostToolUse",
  "tool_name": "TodoWrite",
  "tool_input": {
    "todos": [{ "content": "write a readme", "status": "pending", "id": "1" }]
  },
  "tool_response": {
```

```

    "oldTodos": [],
    "newTodos": [{ "content": "write a readme", "status": "pending", "id": "1" }]
  }
}

```

And for comparison, here's an example of the input to a Stop hook:

```

{
  "session_id": "af9f50b6-f042-4773-b3e2-c3a4814765ce",
  "transcript_path": "<path_to_transcript>",
  "hook_event_name": "Stop",
  "stop_hook_active": false
}

```

As you can see, the stdin input to your command will differ significantly based upon the hook (PreToolUse, PostToolUse, Stop, etc) *and* the matcher used (in the case of PreToolUse and PostToolUse). This can make writing hooks challenging - you might not know the exact structure of the input to your command!

To handle this challenge, try making a helper hook like this:

```

"PostToolUse": [ // Or "PreToolUse" or "Stop", etc
{
  "matcher": "*",
  "hooks": [
    {
      "type": "command",
      "command": "jq . > post-log.json"
    }
  ]
},
]

```

Notice the provided command. It will write the input to this hook to the `post-log.json` file, which allows you to inspect exactly what would have been fed into your command! This makes it a lot easier for you to understand what data your command should inspect.

The Claude Code SDK

Open in Claude

Note: the video above uses an older package name that no longer works. Follow the text below for the current version.

The Agent SDK lets you run Claude Code programmatically from your own applications and scripts. It's available for TypeScript and Python, and gives you the same agent loop the CLI uses — file reading, editing, tool use — under your control.

Install

Create a directory for the project and install the SDK package:

```
mkdir sdk-demo
```

```
cd sdk-demo
```

```
npm init -y
```

```
npm install @anthropic-ai/claude-agent-sdk
```

The package is **@anthropic-ai/claude-agent-sdk**. (The similarly-named **@anthropic-ai/claude-code** is the CLI itself and can't be imported.)

A minimal example

Create a file called **index.mjs** using your editor (or **nano index.mjs** in the terminal) and paste:

```
import { query } from "@anthropic-ai/claude-agent-sdk";
```

```
const prompt = "List the files in the current directory";
```

```
for await (const message of query({ prompt })) {
```

```
  console.log(JSON.stringify(message, null, 2));
```

```
}
```

Run it:

```
node index.mjs
```

You'll see a stream of JSON messages — the same conversation events you'd see in the CLI, including tool calls, tool results, and Claude's text.

Restricting tools

By default the SDK has access to the full tool set. To narrow it, pass **allowedTools**:

```
for await (const message of query({  
  prompt,  
  options: { allowedTools: ["Read", "Glob"] },  
})) {  
  // ...  
}
```

This is the SDK equivalent of the CLI's **--allowedTools** flag.

Where to go next

The SDK supports everything the CLI does: custom system prompts, MCP servers, hooks, subagents, and session resumption. See the [Agent SDK documentation](#) for the full reference.